

Alerting system for remote devices

front: ten minutes a phase · SD Q5 family, alerting half (coach variant)

Anthropic question this is: no bank card asks it word for word (crawl edition 18 has no alerting or device card). It is the **alerting half of SD Q5**: the same Kafka fan-in as the Sep 18 onsite telemetry prompt (§10, crawl Q5) and the 1point3acres data-infrastructure card (§9, Q20), and it builds the alert machinery the analytics dashboard (§11, Q32) left out. **Cousins:** crawl Q40 token-path observability and SLOs · Q11 scale, monitor, debug · the on-call paging lesson. **Tyler graded:** say each box's job, what it reads and what it writes.

1 REQUIREMENTS 0–8 min · five questions, then move

- **Ask:** who gets the alert, on which channels? rules per device or global? a device returns after an hour: do old samples still alert? how fresh? how many devices, numbers per sample, samples per second?
- **A user can:** create, edit and delete a rule (over/under a limit; changed by more than X within Y s) · choose SMS, Slack or email · receive the alert · see open alerts and ack one.
- **Given, not designed:** 128 numbers every second, pre-configured, 6 h buffer on the device. Say it and move on. **Do not design the device.**
- **Technical challenges:** batches arrive late, twice or out of order · **event time, not arrival time** · rate-of-change needs per-device order and state · one dead channel must not block the rest · no storms, no repeats · availability over consistency.

2 CORE ENTITIES 8–14 min · things you can count

- **Device**(id, last_seen, acked_seq)
- **Sample**(device_id, seq, event_ts, values[128])
- **AlertRule**(id, metric_index, op over|under|delta, threshold, window_s, severity, version)
- **Subscription**(rule_id, channel SMS|Slack|email, target)
- **Alert**(id, rule_id, device_id, event_ts, value, state firing|resolved, late)
- **Notification**(alert_id, channel, status, attempts)
- **Not entities:** the metric name (a label on an index, kept in the rule) · the queue (infrastructure).

3 API 14–18 min · semantics as you write

- `POST /devices/{id}/samples` {samples:[{seq, ts, values[128]}]} → 200 {acked_through: seq} | 429 {retry_after}. **Idempotent on (device, seq)**, so a resend is free.
- `POST|GET|PUT|DELETE /rules` · `POST /rules/{id}/subscriptions`
- `GET /alerts?state=firing` · `POST /alerts/{id}/ack`
- Tyler's CRUD test: rules get all four operations; alerts are read and acked; samples are only created.

4 DATA FLOW 18–22 min · one batch end to end

- 1 device sends ~60 samples over TCP → 2 gateway: auth, skip seqs it already has, **append to Kafka, then ack** acked_through →
- 3 rule evaluator consumes by partition (device order kept), updates each device's window, checks rules on **event time** → 4 alert event →
- 5 alert service: dedupe (rule, device, window), firing → resolved, **late** flag, write alert → 6 one message per channel queue →
- 7 sender pulls, calls Twilio / Slack webhook / SES, retries with backoff, DLQ after N → 8 user.
- Side path: gateway updates last_seen; liveness checker raises *device offline* after 10 min of silence.

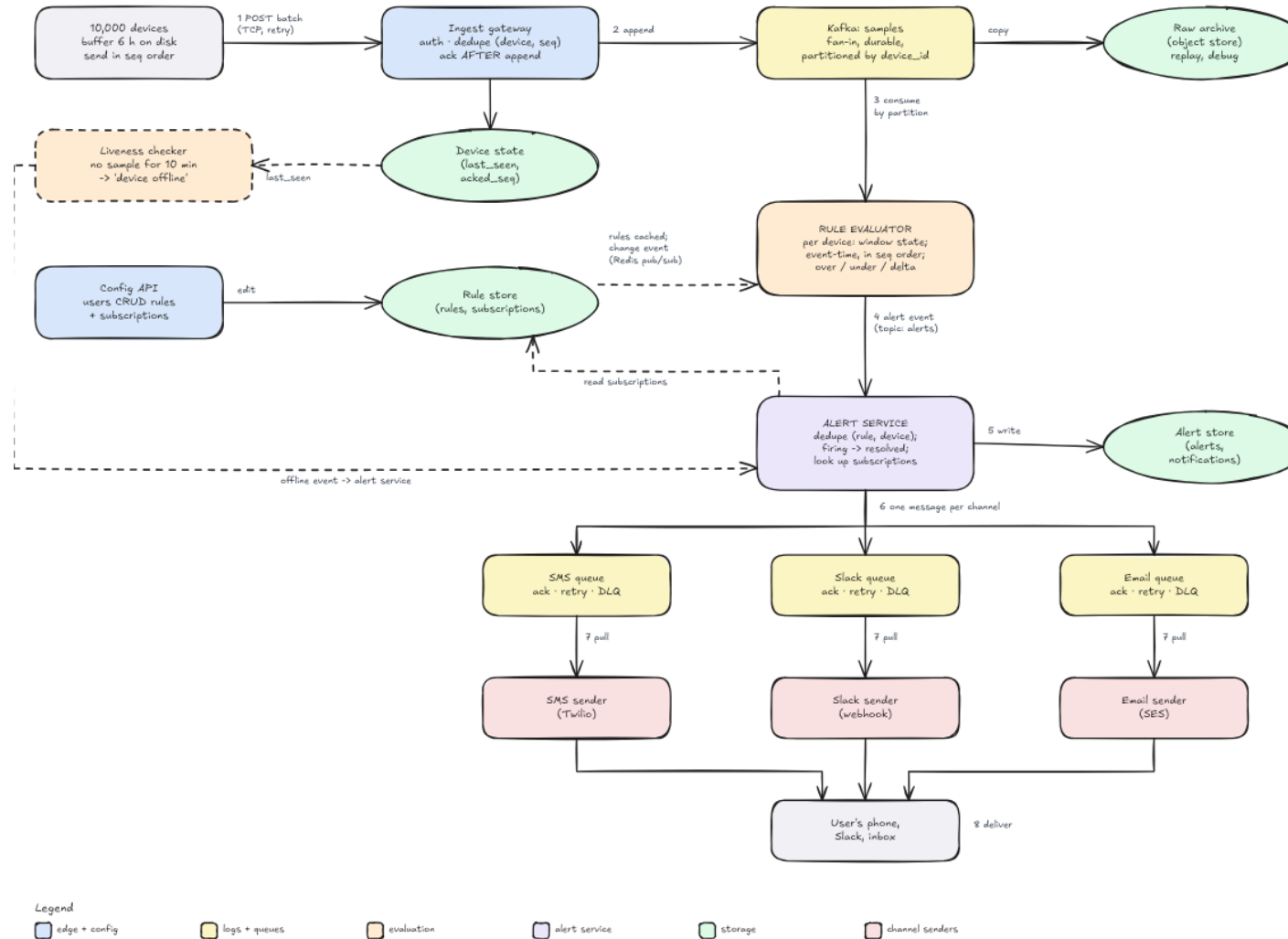
5 HIGH-LEVEL DESIGN 22–32 min · finish here

- **Edge:** devices → ingest gateway → device state
- **Log:** Kafka *samples*, keyed by device_id → raw archive (replay)
- **Evaluation:** rule evaluator (rules cached, Redis pub/sub on change) ← rule store ← config API
- **Alerting:** alert service → alert store; SMS / Slack / email queue → sender → user
- **Sums:** 10k devices × 1/s = 10k samples/s · 128 × 8 B ≈ 1 KB · 10 MB/s ≈ 860 GB/day (**not** 100 MB) · 6 h offline = 21,600 samples ≈ 22 MB, drained at 100× in ~4 min · 100 devices reconnecting ≈ 2× load for minutes.
- **Kafka** = fan-in, durable, replay · **queue** = fan-out of work, ack, retry, DLQ · **Redis pub/sub** = broadcast, in memory, no history.

6 DEEP DIVES 32–45 min · offer, then follow

- **Late data (the point of the prompt):** evaluate on event time; alerts from backlog say *LATE, data from 05:10 UTC*; severity decides whether a late alert pages.
- **Draining:** oldest-first, acked batches of ~1 min, resume from acked_through, per-device cap + 429. Live and backlog share one ordered stream so delta windows stay right.
- **Transport:** TCP (cannot lose samples); short idempotent HTTPS batches beat a WebSocket on a link that drops; MQTT QoS 1 as the named alternative.
- **Per-channel isolation (Tyler):** Slack down → only the Slack queue grows.
- **No storms:** fire once, resolve once, hysteresis; group a rule tripping on 112 devices into one message; **UTC end to end** (Tyler's 2 a.m. DST page).
- **Box test:** for any box, say its job, what it reads, what it writes, what happens when it dies.
- **The bow at 45:** ack after append and dedupe on (device, seq), so resends are free; evaluate on event time and mark late alerts, so a returning device never pages about the past as if it were now.

The board at minute thirty-two alerting system for remote devices — the answer



NUMBERS TO SAY OUT LOUD
 10,000 devices x 1 sample/s = 10,000 samples/s. 128 numbers x 8 B = ~1 KB a sample.
 10,000 x 1 KB = 10 MB/s -> ~860 GB a day raw (not 100 MB: do the multiplication).
 6 h offline = 21,600 samples = ~22 MB per device. Drain at 100x real time = ~4 min.
 Rule evaluation: 10k samples/s x R rules, in memory, per device; trivial for one consumer group.
 Latency budget: seconds; a person takes ~30 s to look at a phone. Availability over consistency.

RED: what fails and what you say
 Late data: evaluate on event time, mark the alert LATE; never page as if it were happening now.
 Duplicates: idempotent on (device_id, seq); alerts deduped on (rule, device, window).
 A channel is down: its queue grows, retries back off; DLQ after N; the other channels are untouched.
 Flapping: fire once, resolve once (hysteresis); severity decides SMS versus Slack.
 Timestamps in UTC end to end. Tyler's 2 a.m. false page was a daylight-saving bug.